

Aim: - To design and implement parallel breadth first search (BFS) and Depth first search (DFS) using openmp and analyze their execution on a tree or undirected graph

Prerequisite -

- (i) knowledge of c/c++ programming
- (ii) understanding of:
 - (a) graph representation (adjacency)
 - (b) BFS and DFS algo, list/matrix
- (iii) Basics of parallel computing
- (iv) Familiarity with openmp directives
- (v) compiler supporting openmp.

Theory -

Graph traversal is a fundamental operation in computer science used in networking, path finding, social networks, etc. Two main traversal technique are

- ① Breadth first search (BFS)
explores level by level
- ② Depth first search (DFS)
explores depth wise

In sequential execution, these algorithms process one node at a time, using openMP, we can parallelize parts of these algorithms to improve performance.

① Breadth first search (BFS) -

① BFS starts from a source node & explores the neighbours first.

② It uses a squared queue data structure.

parallel BFS concept -

① Nodes at the same level can be processed in parallel.

② openMP is used to distribute nodes among threads.

② Depth first search (DFS) -

① DFS explores as far as possible along a branch before backtracking.

② It uses a stack or recursion.

parallel DFS concept -

① Different branches of the graph can be explored simultaneously.

② Care must be taken to avoid race conditions using shared data.

OpenMP Overview -

OpenMP is an API for parallel programming in shared memory systems

Common directives -

- ① #pragma omp parallel
- ② #pragma omp for
- ③ #pragma omp critical

* Algorithm -

Algorithm for parallel BFS :-

- ① start from the source node
- ② mark it as visited and insert into queue.
- ③ while queue is not empty
 - remove a node from queue
 - process all its adjacent nodes in parallel
 - if not visited
 - ① Mark as visited
 - ② Add to queue.

④ Repeat until all nodes are visited.

Conclusion :-

- ① parallel BFS & DFS can significantly improve performance for large graphs
- ② BFS parallelization works well level-wise
- ③ DFS parallelization is more complex due to recursion & shared data.

④. Proper synchronization is necessary to avoid race condition.

Name:- Sarthak Ramdas Ghuge 2200036 comp-n
A.I.S.S.M.S. Batch-C
COLLEGE OF ENGINEERING
NO - 1, Kennedy Road, Near R.T.O Pune - 411001

Experiment :- 2

Aim - To implement parallel bubble sort and parallel merge sort using openmp and compare their performance with sequential algorithm.

Prerequisite-

- (i) knowledge of c/c++ programming
- (ii) understanding of sorting algorithm:
 - (a) bubble sort
 - (b) merge sort
- (iii) Basics of parallel computing
- (iv) knowledge of openmp directives
- (v) GCC compiler with -fopenmp

Theory -

Sorting is one of the most important operations in computer science, efficient sorting improves performance in searching, data processing and analytics.

Two commonly used sorting algorithms:

- (i) Bubble sort: simple but inefficient $O(n^2)$
- (ii) Merge sort: efficient divide and conquer algorithm $O(n \log n)$

parallel computing helps reduce execution time by dividing work among multiple threads.

* *Text*

* Bubble sort -

- repeatedly swaps adjacent elements if they are in wrong order.
- time complexity : $O(n^2)$

* parallel bubble sort (odd-even sort) :-

parallel version is called odd-even Transposition sort.

steps :-

- perform composition in two phases
 - odd phase
 - Even phase

- Each phase can run in parallel using Openmp

* Merge Sort -

- uses divide and conquer
- Divides array into halves, sort them then merged
- time complexity :- $O(n \log n)$

parallel merge sort -

- Each recursive division can run in parallel
- merge sort remains sequential or partially parallel.

* Performance Measurement -

① Use `omp_get_wtimer()` to measure execution time.

② Compare:-

③ Sequential time

④ Parallel time

* Algorithm -

① Sequential bubble sort -

- ① Traverse array from 0 to $n-1$
- ② compare adjacent element
- ③ swap if needed
- ④ Repeat until sorted

② Parallel bubble sort:-

- ① Repeat for n iteration -
 - perform even phase in parallel
 - perform odd phase in parallel

② Continue until sorted

* Conclusion -

- ① parallel algorithm reduce execution time significantly for large datasets.
- ② parallel bubble sort shows improvement but is still inefficient due to $O(n^2)$
- ③ parallel merge sort perform much better due to divide and conquer
- ④ openmp simplifies parallelization using directives -
 - ① Number of threads
 - ② Input size
 - ③ System hardware.

Experiment :- 3

Aim:- To implement min, max, sum and average operations using parallel reduction in openmp and analyze performance.

prerequisites:-

- ① knowledge of c/c++ programming
- ② understanding of
 - a) arrays
 - b) Basics operations
- ③ Basics of parallel computing
- ④ familiarity with openmp directives
- ⑤ gcc compiler with -fopenmp.

Theory -

Reduction is a technique used in parallel computing to combine elements of a dataset into a single result.

Example:-

- a) sum of array
- b) maximum of array
- c) minimum of array
- d) average of array

In sequential execution, operations are performed one by one. In parallel reduction, multiple elements are processed

simultaneously and combined efficiently

* Operations :-

① Sum -

- adds all elements
- Reduction operation : +

② minimum -

- finds smallest element
- uses comparison

③ Maximum -

- finds largest element
- uses comparison

④ Average -

$$\text{Average} = \frac{\text{Sum}}{n}$$

* Algorithm :-

parallel sum :-

① Initial Sum = 0

② Divide array among threads

③ each threads computes partial sum

④ Combine all partial sum.

* parallel min -

- ① initialize min = first element
- ② compare elements in parallel
- ③ update minimum value

* parallel max -

- ① initialize max = first element
- ② compare elements in parallel
- ③ update ~~maximum~~ maximum value.

* Conclusion -

- ① parallel reduction efficiently computes aggregate operations.
- ② openmp reduction clause simplifies implementation.
- ③ Significantly speed improvement is observed for large dataset.
- ④ Avoids race conditions by using thread private variables.

Name :- Sarthak Ramdas Ghuge

22CO036 comp-A

A.I.S.S.M.S

Batch - C

Experiment :- 1

COLLEGE OF ENGINEERING

NO - 1, Kennedy Road, Near R.T.O Pune - 411001

Aim :- To implement CUDA program for

① Addition of two vectors

② Matrix multiplication using CUDA C and analyze parallel execution of GPU

Prerequisite -

① knowledge of C/C++ programming

② Basics of :-

③ vector and matrices

④ parallel computing concepts

⑤ understanding of GPU architecture

⑥ CUDA Toolkit installed

⑦ NVIDIA GPU (for execution)

Theory -

CUDA (compute unified device architecture) is a parallel computing platform by NVIDIA that allows execution of program by GPU for high performance

CPU - sequential execution

GPU - massive parallel execution using thousand of threads.

* CUDA programming model -

① Host (CPU) controls execution

① Device (GPU) - executes kernel

② kernel :- function executed on GPU

* Thread are organized into

① Blocks

② Grids

① Vector Addition -

Each thread add one element -

$$C[i] = A[i] + B[i]$$

parallelism: one thread per element

* Algorithm -

① Initialize vectors A and B

② Allocate memory on GPU

③ Copy data from CPU and GPU

④ Launch kernel

⑤ Each thread computes one element

⑥ copy result back to CPU

② Matrix multiplication -

for matrix A & B

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] * B[k][j]$$

* Algorithm -

- ① Initialize matrices A & B
- ② allocate GPU memory
- ③ Copy data to device
- ④ Launch kernel
- ⑤ Each thread compute one element of result.
- ⑥ copy result back

* Conclusion :-

- ① CUDA enables massive parallelism using GPU
- ② Vector addition is simplest & highly parallel
- ③ Matrix multiplication benefits greatly from GPU architecture
- ④ CUDA is widely used in AI, ML and Scientific computing.